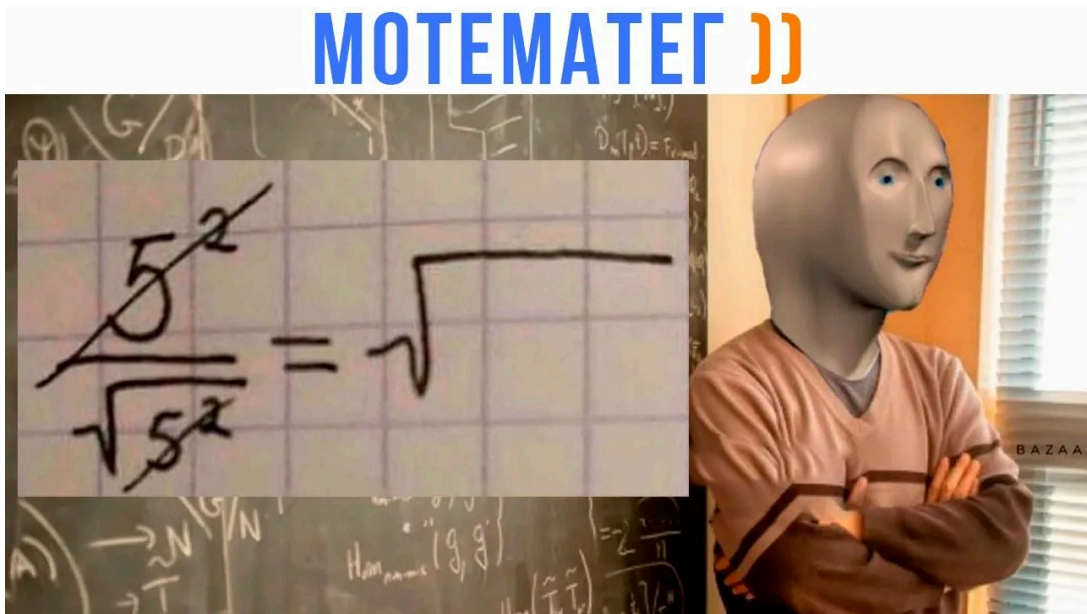


Математическое моделирование физических процессов

Лектор: Горюнов Алексей Германович
Преподаватель (ПР, ЛБ): Суханов Евгений Андреевич

В основном рассматривается Python, поэтому всё что связано с Matlab смотреть в [лекциях](#).
Код в данном конспекте представлен частично, полную его версию можно также найти на [GitHub](#).



Всё, что на ПК, сюда попадает

Классификация модели по "абстракции":

- Материальная
 - Геометрическая
 - Физическая
 - Аналоговая
- Абстрактная
 - Мнемоническая
 - Вычислительная ! одно и тоже !
 - Компьютерная
 - Математическая:

Признак	МатМодель
Характер отображения свойств объекта	Структурные, Функциональные
Иерархический уровень	Микро-, Макро-, Метауровень
Степень детализации описания внутри одного уровня	Полные, Макромодели
Способ представления свойств объекта	Аналитические, Алгоритмические, Имитационные
Способ получения	Теор., Эмпир.
...	...

Аналитическая модель – математическая модель, представляющая собой совокупность **аналитических** выражений и зависимостей, позволяющих оценивать те или иные свойства моделируемого объекта. **Аппроксимация по значениям, таблицам**

Имитационная модель – универсальное средство исследования сложных систем, представляющее собой **логику-алгоритмическое описание** поведения отдельных элементов системы и правил их взаимодействия, отображающих последовательность событий, возникающих в моделируемой системе. **Всё что связано с частицами; метод Монте-Карло**

1.1.4 Подход к математическому моделированию процессов

7



Info >

Записал определение, сделал ссылочку на рецензируемое
Схему нужно пройти от и до для выполнения курсового проекта

1.2 Введение в Matlab. Язык Matlab: константы и переменные

Константа – это предварительно определенное числовое или символьное значение, представленное уникальным именем.

Числа являются безымянными числовыми константами. Другие виды констант в MATLAB принято называть системными переменными, поскольку, с одной стороны, они задаются системой при ее загрузке, а с другой – могут переопределяться. Основные системные переменные, применяемые в системе MATLAB, указаны ниже:

- **i** или **j** – мнимая единица;
- **pi** – число π – 3.1415926...;
- **eps** – погрешность операций над числами с плавающей точкой (2⁻⁵²);
- **realmin** – наименьшее число с плавающей точкой (2⁻¹⁰²²);
- **realmax** – наибольшее число с плавающей точкой (2¹⁰²³);
- **inf** – значение машинной бесконечности;
- **ans** – переменная, хранящая результат последней операции и обычно вызывающая его отображение на экране дисплея;
- **NaN** – указание на нечисловой характер данных (Not-a-Number).

Основные типы данных: числовые, logical, char, cell, struct, table и function_handle.

1.3 Введение в Python для задач моделирования

Переменная – это область памяти, в которой хранятся наши данные. В переменную задается имя и значение, например:

```
test = 1
value = "2"
result = 3.0
test2 = True
```

Каждая переменная содержит тип данных, например: `float`, `int`, `str`, `bool`, `complex` и т.д. Арифметические операторы используются с числовыми значениями для выполнения общих математических операций: `+`, `-`, `*`, `/`, `%`, `**`, `//`.

Операторы присваивания используются для присваивания значений переменным: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `//=`, `**=`, `&=`, `|=`, `^=`, `>=`, `<=`.

Операторы сравнения, логические: `==`, `!=`, `>`, `<`, `>=`, `<=`, `and`, `or`, `not`.



```
# Условный оператор
if condition:
    <indented statement 1>
    <indented statement 2>
elif condition:
    <indented statement 3>
else:
    <indented statement 4>

# Циклы: оператор for
for <variable> in <sequence>:
    <body of for>
else:
    <body of else>
```

```
# Циклы: оператор while
while test_expression:
    body of while
```

Лекция 2. Работа с данными в Matlab и Python

2.1 Операции с массивами/матрицами в Python

Доступ к элементам матриц осуществляется при помощи двух индексов - номеров строки и столбца, заключенных в квадратные скобки для каждого индекса отдельно. [БАЗА. При этом нумерация идёт с 0](#)

При помощи библиотеки NumPy вектора/матрицы записываются так:

```
import numpy as np

# ----- Создание матриц -----
A = np.array([[1, 2, 3],
              [1, 2, 3],
              [6, -2, 8]])          # матрица 3x3

B = np.array([[8, 8, 8],
              [1, 2, 3],
              [-6, 2, -8]])        # матрица 3x3

# ----- Специальные матрицы -----
Zer = np.zeros((3, 3))            # нулевая матрица 3x3
Eye = np.eye(3)                   # единичная матрица 3x3
One = np.ones((3, 3))             # матрица из единиц 3x3
R = np.random.rand(3, 3)          # случайная матрица [0,1)
Diag = np.diag([1, 2, 3])         # диагональная матрица diag(1,2,3)

# ----- Последовательности и reshaping -----
v = np.arange(0, 10, 1)           # [0,1,2,3,4,5,6,7,8,9]
M = np.arange(1, 10).reshape(3, 3) # [[1,2,3],[4,5,6],[7,8,9]]
linspace_mat = np.linspace(0, 1, 9).reshape(3, 3) # равномерная сетка 0..1

# ----- Базовые арифметические операции -----
S = A + B                         # поэлементная сумма
D = A - B                         # поэлементная разность
P_elem = A * B                    # поэлементное умножение
P_mat = A @ B                     # матричное умножение
P_mat2 = np.dot(A, B)             # то же самое
G = A * 3                        # умножение на число
Div = A / 2.0                    # деление на число
Pow = A ** 2                     # поэлементное возведение в квадрат

# ----- Транспонирование и перестановка осей -----
T = B.T                           # транспонирование B
T2 = np.transpose(B)              # то же самое
C = np.rot90(A)                   # поворот A на 90° против часовой
C2 = np.rot90(A, k=2)             # поворот A на 180°
```

```

# ----- Обратная матрица и определитель -----
detB = np.linalg.det(B)          # определитель матрицы B
invB = np.linalg.inv(B)          # обратная матрица к B (если det ≠ 0)
# Проверка: B @ invB ≈ Eye

# ----- Решение СЛАУ Ax = b -----
A_slau = np.array([[1, 2, 3],
                   [0, 1, 4],
                   [5, 6, 0]], dtype=float)
b = np.array([1, 2, 3], dtype=float)
x = np.linalg.solve(A_slau, b)    # решение системы Ax = b

# ----- Индексация и срезы -----
# Доступ к элементу: строка i, столбец j (нумерация с 0)
a11 = A[0, 0]                    ''' Вывод: 1 '''
a23 = A[1, 2]                    ''' Вывод: 3 '''

# Строки и столбцы
row0 = A[0, :]                   ''' Вывод: [1, 2, 3] '''
row2 = A[2, :]                   ''' Вывод: [ 6, -2,  8] '''
col1 = A[:, 1]                   ''' Вывод: [ 2,  2, -2] '''

# Срезы (подматрицы)
sub = A[0:2, 1:3]                ''' Вывод: [[ 2,  3],
                                     [ 2,  3]] '''
sub2 = A[:2, 1:]                 ''' Вывод: [[ 2,  3],
                                     [ 2,  3]] '''

# Отрицательные индексы (с конца)
last_row = A[-1, :]              ''' Вывод: [ 6, -2,  8] '''
last_col = A[:, -1]              ''' Вывод: [3, 3, 8] '''
corner = A[-1, -1]               ''' Вывод: 8 '''

# Продвинутая (fancy) индексация
idx_rows = [0, 2]
idx_cols = [1, 2]
vals = A[idx_rows, idx_cols]     ''' Вывод: [2, 8] (A[0,1] и A[2,2]) '''

# Булева индексация (условия)
mask = A > 2                     ''' Вывод: матрица True/False,
                                     где A[i,j] > 2 '''
A_big = A[A > 2]                 ''' Вывод: [3, 3, 6, 8] (все элементы > 2) '''
A[A < 0] = 0                     ''' A после: отрицательные элементы заменены на 0
'''

```

...

2.2 Основные библиотеки Python для моделирования

[NumPy](#) – библиотека для вычислений многомерных и просто крупных массивов, включает множество функций, которые позволяют эффективно проводить операции и выполнять высокоуровневые математические расчеты.

[matplotlib](#) – библиотека, предназначенная для отображения двумерных и 3D-графиков, максимально точно воссоздает среду MATLAB в плане графического интерфейса и синтаксической формы.

[SciPy](#) – основная библиотека для научных вычислений в Python, содержащая множество функций для численного решения дифференциальных уравнений, включая методы, такие как метод Эйлера и метод Рунге-Кутты, также статистические расчеты, интерполяцию и аппроксимацию данных (в дополнение к NumPy).

[SymPy](#) – библиотека для выполнения символьных вычислений

2.3 Импорт и экспорт данных в Python

[То что ненавижу в работе с Python](#)((.

```
'''Встроенная функция open(). С ее помощью можно открыть любой файл на компьютере.
Python создает на его основе объект.'''
f = open(file_name, access_mode)

'''Функция read() используется для чтения содержимого файла после открытия его в
режиме чтения (r).
    • file = объект файла
    • size = количество символов, которые нужно прочитать. Если не указать, то файл
прочитается целиком. '''
file.read(size)

''' Функция write() используется для записи в файлы Python, открытые в режиме записи.
Если попытаться открыть файл, которого не существует, в этом режиме, тогда будет создан
новый.'''
file.write(string)

'''Метод close(). После открытия файла в Python его нужно закрыть. Таким образом
освобождаются ресурсы и убирается мусор. Python автоматически закрывает файл, когда
объект присваивается другому файлу.'''
```

2.4 Функции в Python

Пока в Matlab существуют m-файлы, анонимные, вложенные функции, подфункции и операторы дескриптора функции, в Python есть просто функции))). def , lambda ... которые могут ещё использоваться для создания классов в ООП.

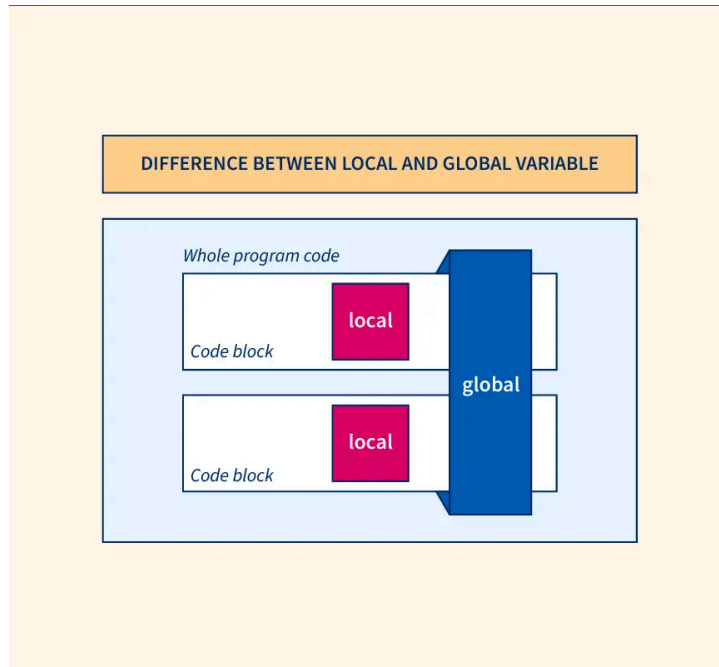
```
# ----- Стандартные функции -----
def name_function(var1, var2, ...): # или без аргументов: def name_function():
    '''docstring – краткое описание функции'''
    <instruction 1>
    <instruction 2>
    <instruction 3>
    return expression                # возвращаемое значение (может быть несколько
через кортеж)
```

```
# ----- Лямбда-функции (анонимные функции) -----  
x = lambda var: expression      # одна строка, одно выражение, неявный return
```

2.5 Локальные и глобальные переменные

Локальные переменные — объявляются внутри функции (в том числе аргументы). Видны только внутри этой функции.

Глобальные переменные — объявляются вне функций (на «верхнем» уровне модуля). Видны везде в этом файле, если не перекрыты локальными с тем же именем.



```
# ----- Локальные и глобальные переменные -----  
g = 10      # глобальная переменная (объявлена вне функций)  
def f(x):   # x — локальный аргумент  
    y = x + g  # g берётся из глобальной области  
    return y  
print(f(5))  # 15  
print(g)    # 10  
  
# Изменение глобальной переменной внутри функции  
g = 10  
def change_global():  
    global g      # явно указываем, что g — глобальная  
    g = g + 5  
change_global()  
print(g)         # 15
```

Лекция 3. Визуализация вычислений в Matlab и Python

3.1 Графическая визуализация вычислений в Python

[matplotlib](#) — библиотека, предназначенная для отображения двумерных и 3D-графиков, максимально точно воссоздает среду MATLAB в плане графического интерфейса и

синтаксической формы. [12](#)

3.2 Построение двумерных графиков в Python

[Старт в Matplotlib](#)

Пример 1

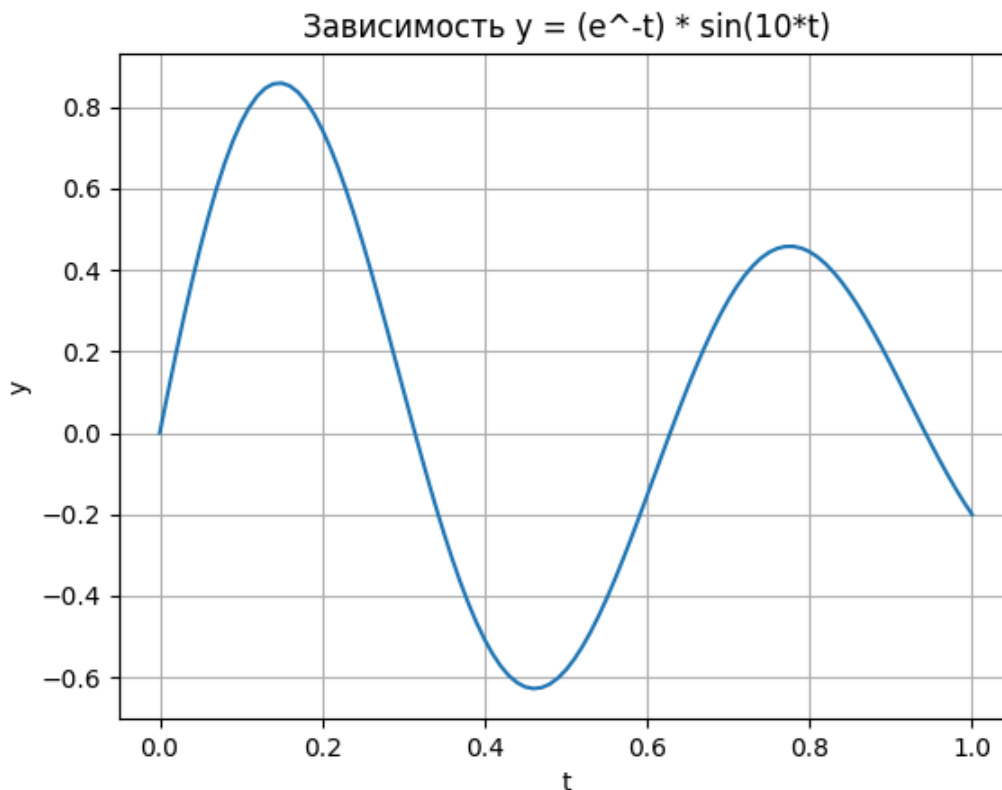
График функции $y(t) = e^{-t} \cdot \sin(10 \cdot t)$, где аргумент t меняется в интервале $[0, 1]$ с шагом $0,01$.

```
import matplotlib.pyplot as plt
import numpy as np

t = np.zeros(101)
y = np.zeros(101)
y[0] = np.exp(-t[0]) * np.sin(10*t[0])

for i in range(1,101):
    t[i] = t[i-1] + 0.01
    y[i] = np.exp(-t[i]) * np.sin(10*t[i])

# Построение графика
plt.title("Зависимость y = (e^-t) * sin(10*t)") # заголовок
plt.xlabel("t") # ось абсцисс
plt.ylabel("y") # ось ординат
plt.grid() # включение отображение сетки
plt.plot(t, y) # построение графика
plt.show()
```



Пример 2

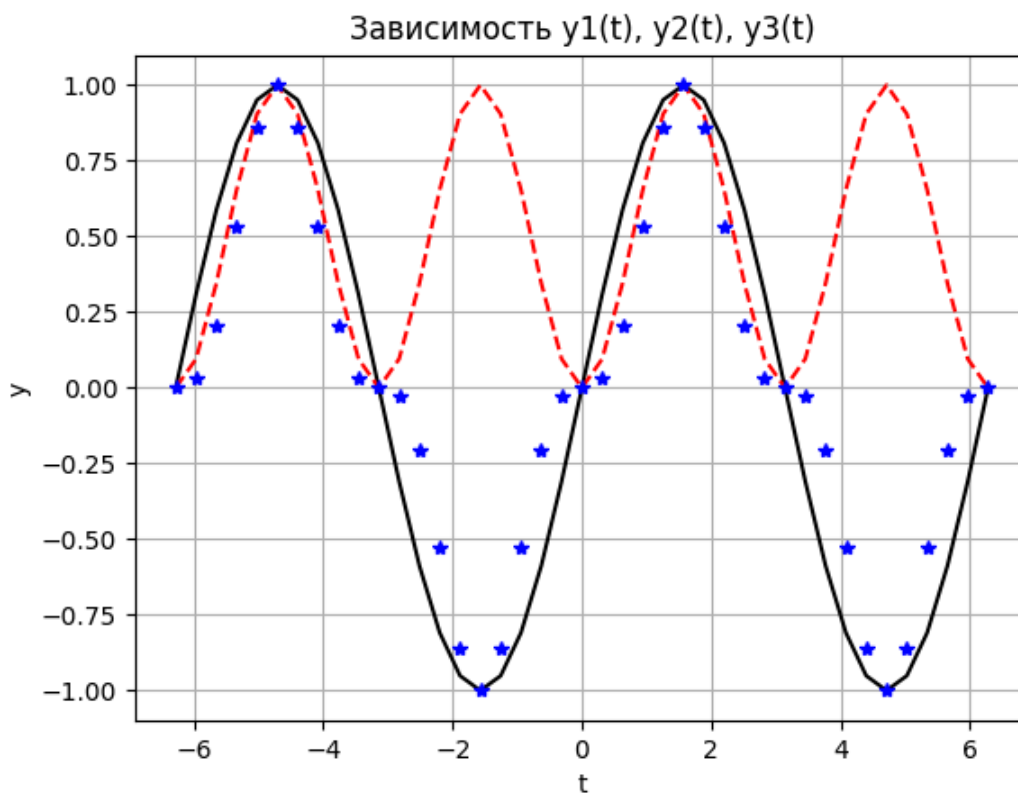
График функции $y_1(t) = \sin(t)$, $y_2(t) = \sin^2(t)$ и $y_3(t) = \sin^3(t)$ при изменении t в интервале $[-2\pi, 2\pi]$ с шагом $0,1\pi$.

```
import matplotlib.pyplot as plt
import numpy as np

t = np.zeros(41)
t[0] = -2*np.pi
y1 = np.zeros(41)
y2 = np.zeros(41)
y3 = np.zeros(41)
y1[0] = np.sin(t[0])

for i in range(1,41):
    t[i] = t[i-1] + 0.1*np.pi
    y1[i] = np.sin(t[i])
    y2[i] = pow(np.sin(t[i]),2)
    y3[i] = pow(np.sin(t[i]),3)

# Построение графика
plt.title("Зависимость y1(t), y2(t), y3(t)")
plt.xlabel("t") # ось абсцисс
plt.ylabel("y") # ось ординат
plt.grid() # включение отображение сетки
plt.plot(t, y1, "k-", t, y2, "r--", t, y3, "b*")
plt.show()
```



Пример 3

График функции $z(x, y) = x + y^3$, где переменные x и y изменяются на интервале $[-10, 10]$ с шагом 0,5. Для построения графика используем непрерывные линии черного цвета.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.zeros(41)
x[0] = -10
for i in range(1,41):
    x[i] = x[i-1] + 0.5

y = x

# function for z axis
def f(x, y):
    return x + pow(y,3)

X, Y = np.meshgrid(x, y)
Z = f(X, Y)

fig = plt.figure()
ax = plt.axes(projection = '3d')
ax.plot_wireframe(X, Y, Z, color = 'green')
ax.set_title("z(x,y) = x + y^3")
ax.set_xlabel('x', fontsize=12)
ax.set_ylabel('y', fontsize=12)
ax.set_zlabel('z', fontsize=12)
plt.show()
```

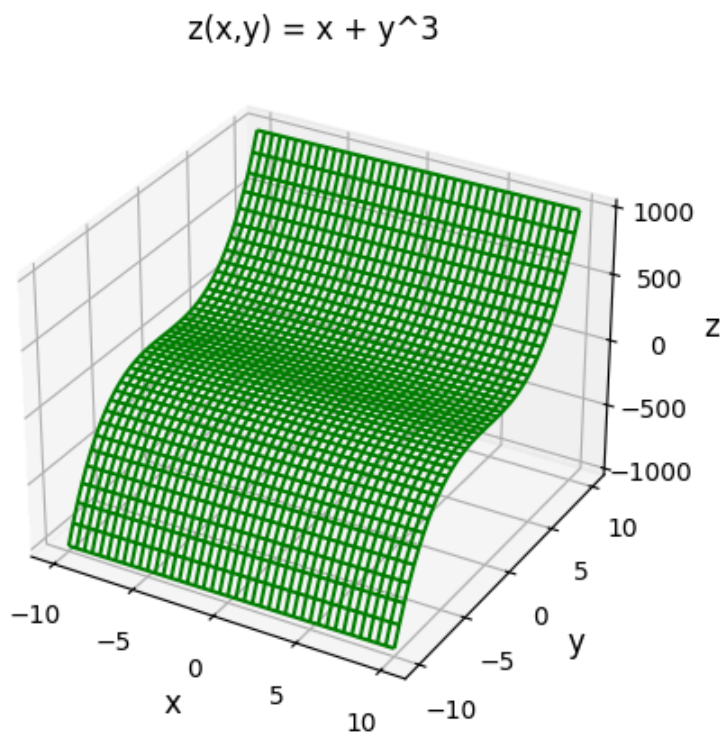


График функции $z = \frac{\sin(x)}{x^2+y^2+0.3}$, где x и y изменяются в интервале $[-4, 4]$ с шагом 0,1.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.zeros(81)
x[0] = -4
for i in range(1,81):
    x[i] = x[i-1] + 0.1

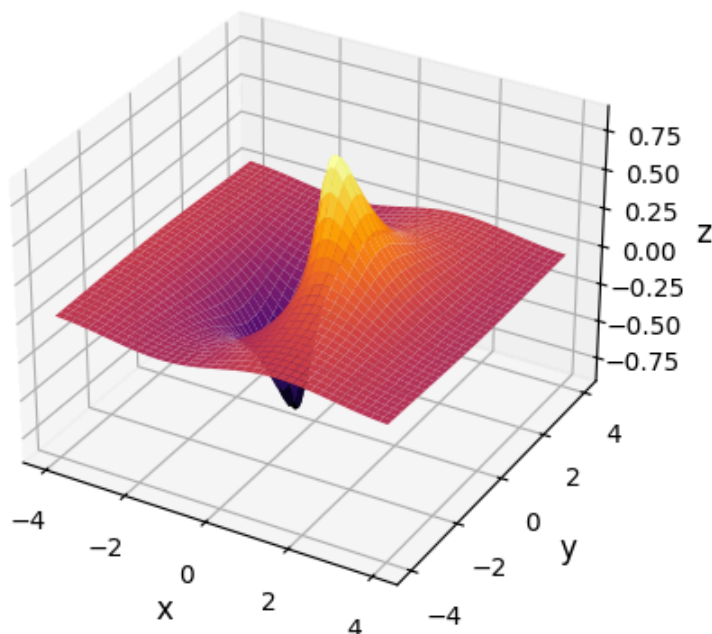
y = x

# function for z axis
def f(x, y):
    return np.sin(x) / (pow(x,2) + pow(y,2) + 0.3)

X, Y = np.meshgrid(x, y)
Z = f(X, Y)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, cmap='inferno')
ax.set_title("z(x,y)=sin(x)/(x^2 + y^2 + 0.3)")
ax.set_xlabel('x', fontsize=12)
ax.set_ylabel('y', fontsize=12)
ax.set_zlabel('z', fontsize=12)
plt.show()
```

$$z(x,y)=\sin(x)/(x^2 + y^2 + 0.3)$$



Лекция 4. Методы восстановления эмпирических зависимостей

Начиная с этой лекции польётся говно по трубам

4.1 Интерполяция, экстраполяция

Интерполяция – способ нахождения промежуточных значений величины по имеющемуся дискретному набору известных значений. Нахождение приближенной функции. $x_0, x_1, \dots, x_n$ – узлами интерполяции.

Графически задача интерполирования заключается в том, чтобы построить такую интерполирующую функцию, которая бы проходила через все узлы интерполирования.

Вид канонического полинома степени n :

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1} + a_nx^n \quad (1)$$

Выбор многочлена степени n основан на том факте, что через $n+1$ точку проходит единственная кривая степени n . На основе (1) и данных интерполяции составим СЛАУ, решая её, можно найти коэффициенты интерполяционного полинома $a_0, a_1, \dots, a_n$.

Экстраполяция – это индуктивное нахождение по ряду данных значений функции других её значений, находящихся вне этого ряда. **Т.е. приближенное значение $f(x)$, где x лежит вне $[x_0, x_n]$.**

Экстраполяция функции – продолжение функции за пределы её области определения, при котором продолженная функция (как правило, аналитическая) принадлежит заданному классу функций.

4.2 Интерполяционный полином Лагранжа

Интерполяционный полином Лагранжа имеет вид:

$$P_n(x) = \sum_{i=0}^n x_i \cdot L_n(x), \quad (2)$$

где $L_n(x)$ – множитель Лагранжа, определяемый выражением:

$$L_n(x) = \frac{(x - x_0) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)} = \prod_{\substack{k=0 \\ k \neq i}}^n \frac{(x - x_k)}{(x_i - x_k)} \quad (3)$$

В свернутом виде:

$$P_n(x) = \sum_{i=0}^n y_i \left(\prod_{\substack{k=0 \\ k \neq i}}^n \frac{x - x_k}{x_i - x_k} \right) \quad (4)$$

4.3 Интерполяционные полиномы Ньютона

Если узлы интерполяции равноотстоящие по величине, так что выполняется условие:

$$x_{i+1} - x = h = \text{const}, \quad (5)$$

где h – шаг интерполяции, т.е. $x_i = x_0 + i \cdot h$, то интерполяционный многочлен можно записать в форме, предложенной Ньютоном. Интерполяционные полиномы Ньютона как правило используют, если точки интерполирования находится в начале таблицы – **первая интерполяционная формула** Ньютона или в конце таблицы – **вторая формула**.

Интерполирующий полином:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n(x - x_0) \dots (x - x_{n-1}) \quad (6)$$

Коэффициенты $a_0, a_1, \dots, a_n$ находятся из условия $P_n(x_i) = y_i$.

Определим a_0 , полагая $x = x_0$:

$$a_0 = P_n(x_0) = y_0 \quad (7)$$

Для определения a_2 , полагая $x = x_2$, получим:

$$\begin{aligned} P_n(x_2) = y_2 = y_0 + \frac{\Delta y_0}{h}(x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1) &= y_0 + 2\Delta y_0 + a_2 \cdot 2 \cdot h^2, \\ a_2 = \frac{y_2 - y_0 - 2\Delta y_0}{2h^2} = \frac{y_2 - y_0 - 2y_1 + 2y_0}{2h^2} = \frac{y_2 - 2y_1 + y_0}{2h^2} &= \frac{(y_2 - y_1) - (y_1 - y_0)}{2h^2} = \\ &= \frac{\Delta y_1 - \Delta y_0}{2h^2} = \frac{\Delta^2 y_0}{2! h^2}. \end{aligned} \quad (8)$$

Общая формула для нахождения всех коэффициентов имеет вид:

$$\begin{aligned} a_i &= \frac{\Delta^i y_0}{i! h^i}, \\ \Delta^i y_0 &= \Delta^{i-1} y_1 - \Delta^{i-1} y_0, \\ \Delta^i y_1 &= \Delta^{i-1} y_2 - \Delta^{i-1} y_1 \end{aligned} \quad (9)$$

где $i = 1 \dots n$.

$\Delta^2 y_0 = \Delta y_1 - \Delta y_0,$	$\Delta^2 y_1 = \Delta y_2 - \Delta y_1,$
$\Delta y_0 = y_1 - y_0,$	$\Delta y_1 = y_2 - y_1.$

В результате (6) примет вид:

$$\begin{aligned} P_n(x) &= y_0 + \frac{\Delta y_0}{1! h}(x - x_0) + \frac{\Delta^2 y_0}{2! h^2}(x - x_0)(x - x_1) + \dots \\ &\dots + \frac{\Delta^n y_0}{n! h^n}(x - x_0)(x - x_1) \dots (x - x_{n-1}). \end{aligned} \quad (10)$$

бббrrrrr... чё так страшно... плак, плак, т.к это ещё не всё

Для нахождения значений функции **в конце интервала** интерполирования полином Ньютона записывается в виде:

$$\begin{aligned} P_n(x) &= a_0 + a_1(x - x_n) + a_2(x - x_n)(x - x_{n-1}) + \dots \\ &\dots + a_n(x - x_n)(x - x_{n-1}) \dots (x - x_1). \end{aligned} \quad (11)$$

Коэффициенты $a_0, a_1, \dots, a_n$ находятся из условия $P_n(x_i) = y_i$.

... (типо решаем) ...

4.4 Интерполяция сплайнами

Пусть отрезок $[a, b]$ разбит точками $a = x_0 < x_1 < \dots < x_n = b$ на n частичных отрезков $[x_{i-1}, x_i]$.

Сплайном степени m называется функция $S_m(x)$, обладающая следующими свойствами:

1. Функция $S_m(x)$ непрерывна на отрезке $[a, b]$ вместе со всеми своими производными $S_m^{(1)}(x), S_m^{(2)}(x), \dots, S_m^{(p)}(x)$ до некоторого порядка p ;
2. На каждом частичном отрезке $[x_{i-1}, x_i]$ функция $S_m(x)$ совпадает с некоторым алгебраическим многочленом $P_{m,i}(x)$ степени m .

Дефект сплайна = $m - p$.

Наибольшее распространение на практике получили сплайны $S_3(x)$ 3-й степени (кубические сплайны) с дефектом 1 или 2. Такие сплайны на каждом из частичных отрезков $[x_{i-1}, x_i]$ совпадают с кубическим многочленом:

$$S_3(x) = P_{3,i}(x) = a_i + b_i(x - x_{i-1}) + c_i(x - x_{i-1})^2 + d_i(x - x_{i-1})^3 \quad (14)$$

и имеют на отрезке $[a, b]$ по крайней мере одну непрерывную производную $S'_3(x)$.

Значение $s_i = S'_m(x_i)$ называется **наклоном сплайна** в точке x_i . На отрезке $[x_{i-1}, x_i]$ интерполяционный кубический сплайн однозначно определяется заданием значений $y_{i-1}, y_i, s_{i-1}, s_i$ и справедлива следующая формула:

$$S_3(x) = P_{3,i}(x) = \frac{(x - x_i)^2(2(x - x_{i-1}) + h_i)}{h_i^3} y_{i-1} + \frac{(x - x_{i-1})^2(2(x_i - x) + h_i)}{h_i^3} y_i + \\ + \frac{(x - x_i)^2(x - x_{i-1})}{h_i^2} s_{i-1} + \frac{(x - x_{i-1})^2(x - x_i)}{h_i^2} s_i. \quad (15)$$

где $h_i = x_i - x_{i-1}$.

Если в точках x_i известны значения производной $y'_i = f'(x_i)$, то естественно положить $s_i = y'_i$ для всех $i = 0, 1, \dots, n$. Тогда на каждом частичном отрезке $[x_{i-1}, x_i]$ в соответствии с формулой (15) сплайн однозначно определяется значениями $y_{i-1}, y_i, y'_{i-1}, y'_i$ (из-за этого его называют **локальным сплайном**).

Условие непрерывности 2-й производной: $P''_{3,i}(x_i) = P''_{3,i+1}(x_i)$.

Система для наклонов s_i :

$$h_i^{-1} s_{i-1} + 2(h_i^{-1} + h_{i+1}^{-1}) s_i + h_{i+1}^{-1} s_{i+1} = 3 [h_i^{-2} (y_i - y_{i-1}) + h_{i+1}^{-2} (y_{i+1} - y_i)], \quad (16) \\ i = 1, \dots, n - 1.$$

Недоопределена ($n - 1$ уравнений, $n + 1$ неизвестных) $\rightarrow$ нужны **граничные условия**.

Типы граничных условий:

Условие	Уравнения	Название сплайна
$s_0 = f'(a), s_n = f'(b)$	(17)	Фундаментальный
$S''_3(a) = f''(a), S''_3(b) = f''(b)$	(18)	—
$f''(a) = f''(b) = 0$	частный случай (18)	Естественный

4.5 Аппроксимация. Метод наименьших квадратов

4.5.1 Понятие аппроксимации

Аппроксимация – метод приближения, при котором для нахождения дополнительных значений, отличных от табличных данных, приближенная функция проходит не через узлы интерполяции, а между ними.

Метод	Характер
Интерполяция	"...функция строго проходит через узловые точки..."
Аппроксимация	"...приближенная функция проходит не через узлы интерполяции, а между ними."

Если аналитическое выражение функции, описывающей закон изменения неизвестно или весьма сложно, то возникает задача найти такую эмпирическую формулу значения которой при x_i мало отличались бы от экспериментальных данных. Геометрически задача – проведении усредненной кривой.

🔗 Зачем нужна аппроксимация, если есть точная интерполяция?

1. При значительном количестве табличных данных (интерполирующая функция становится громоздкой) (Феномен Рунге);
2. Интерполирующей функцией невозможно описать данные при повторении эксперимента в одних тех же начальных условиях; 1
3. Для сглаживания погрешностей эксперимента;
4. Интерполяция за пределами таблицы (экстраполяция) даёт непредсказуемый результат — полином «улетает». Аппроксимирующая кривая ведёт себя предсказуемо, т.к. её степень фиксирована.

4.5.2 Метод наименьших квадратов

Суть метода: найти такие коэффициенты a_i , при которых сумма квадратов отклонений $e_i = y_i - f(x_i)$ минимальна:

$$\sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - f(x_i))^2 \rightarrow \min \quad (17)$$

Вводим функцию:

$$(a_0, a_1, \dots, a_n) = \sum_{i=1}^n (y_i - f(x_i))^2 \quad (18)$$

Условие минимума — равенство нулю частных производных (нормальная система уравнений):

(19)

$$\begin{aligned} \frac{d}{da_0} &= 0 \\ \frac{d}{da_1} &= 0 \\ \dots \\ \frac{d}{da_n} &= 0 \end{aligned}$$

4.5.3 Линейная аппроксимация методом наименьших квадратов

Применяется, когда данные можно описать линейной зависимостью:

$$P(x) = a \cdot x + b \quad (20)$$

Подставляем (20) в (18):

$$= \sum_{i=1}^n (y_i - a \cdot x_i - b)^2 \min \quad (21)$$

Система из двух уравнений с двумя неизвестными:

$$(22)$$

$$\begin{aligned} \frac{d}{da} &= 0 \\ \frac{d}{db} &= 0 \end{aligned}$$

Раскрываем производные:

$$(23)$$

$$\begin{aligned} \frac{d}{db} &= -2 \cdot \sum_{i=1}^n (y_i - a \cdot x_i - b) \cdot 1 = 0 \\ \frac{d}{da} &= -2 \cdot \sum_{i=1}^n (y_i - a \cdot x_i - b) \cdot x_i = 0 \end{aligned}$$

После группировки относительно коэффициентов:

$$(24)$$

$$\begin{aligned} a \sum_{i=1}^n x_i^2 + b \sum_{i=1}^n x_i &= \sum_{i=1}^n (x_i \cdot y_i) \\ a \sum_{i=1}^n x_i + n \cdot b &= \sum_{i=1}^n y_i \end{aligned}$$

Формулы для коэффициентов a и b :

$$\begin{aligned} a &= \frac{n \cdot \sum_{i=1}^n (x_i \cdot y_i) - \sum_{i=1}^n x_i \cdot \sum_{i=1}^n y_i}{n \cdot \sum_{i=1}^n x_i^2 - \left(\sum_{i=1}^n x_i \right)^2}, \\ b &= \frac{\sum_{i=1}^n y_i \cdot \sum_{i=1}^n x_i^2 - \left(\sum_{i=1}^n x_i \right) \sum_{i=1}^n (x_i \cdot y_i)}{n \cdot \sum_{i=1}^n x_i^2 - \left(\sum_{i=1}^n x_i \right)^2} \end{aligned} \quad (25)$$

4.5.2 Параболическая аппроксимация

Модель — полином 2-й степени:

$$P_2(x) = a_0 + a_1x + a_2x^2 \quad (26)$$

Функция минимизации:

$$= \sum_{i=1}^n (y_i - a_0 - a_1x_i - a_2x_i^2)^2 \min \quad (27)$$

Нормальная система (три уравнения):

(28)

$$\frac{d}{da_0} = -2 \sum_{i=1}^n (y_i - a_0 - a_1x_i - a_2x_i^2) \cdot 1 = 0$$

$$\frac{d}{da_1} = -2 \sum_{i=1}^n (y_i - a_0 - a_1x_i - a_2x_i^2) \cdot x_i = 0$$

$$\frac{d}{da_2} = -2 \sum_{i=1}^n (y_i - a_0 - a_1x_i - a_2x_i^2) \cdot x_i^2 = 0$$

Группировка относительно коэффициентов:

(29)

$$a_0 \cdot n + a_1 \cdot \sum_{i=1}^n x_i + a_2 \cdot \sum_{i=1}^n x_i^2 = \sum_{i=1}^n y_i$$

$$a_0 \cdot \sum_{i=1}^n x_i + a_1 \cdot \sum_{i=1}^n x_i^2 + a_2 \cdot \sum_{i=1}^n x_i^3 = \sum_{i=1}^n (x_i \cdot y_i)$$

$$a_0 \cdot \sum_{i=1}^n x_i^2 + a_1 \cdot \sum_{i=1}^n x_i^3 + a_2 \cdot \sum_{i=1}^n x_i^4 = \sum_{i=1}^n (x_i^2 \cdot y_i)$$

Обозначения сумм:

$$S_1 = \sum_{i=1}^n x_i; \quad S_2 = \sum_{i=1}^n x_i^2; \quad S_3 = \sum_{i=1}^n x_i^3; \quad S_4 = \sum_{i=1}^n x_i^4;$$

$$S_5 = \sum_{i=1}^n y_i; \quad S_6 = \sum_{i=1}^n (x_i \cdot y_i); \quad S_7 = \sum_{i=1}^n (x_i^2 \cdot y_i).$$

Итоговая система для поиска a_0, a_1, a_2 :

$$\begin{aligned}a_0 \cdot n + a_1 \cdot S_1 + a_2 \cdot S_2 &= S_5 \\a_0 \cdot S_1 + a_1 \cdot S_2 + a_2 \cdot S_3 &= S_6 \\a_0 \cdot S_2 + a_1 \cdot S_3 + a_2 \cdot S_4 &= S_7\end{aligned}$$

4.6 Математическая обработка данных в Python

Итак первые 3 пункта тут будут о том, как работать с файлами и данными из них, если вам нужен сам код для интерполяции смотрите начиная с четвёртого пункта.

Также по [ссылке на GitHub](#) будут представлены все варианты кодов.

Получение исходной таблицы данных

$$\begin{aligned}y &= 0.5 \cdot \sin(x) + 2 \cdot \text{an}, \\x &\in [0; 10], \text{ шаг } 1\end{aligned}$$

```
import matplotlib.pyplot as plt
import numpy as np

# --- Часть 1: Получение исходной таблицы данных ---

x = np.zeros(11)
y = np.zeros(11)
float_rn = np.zeros(11)

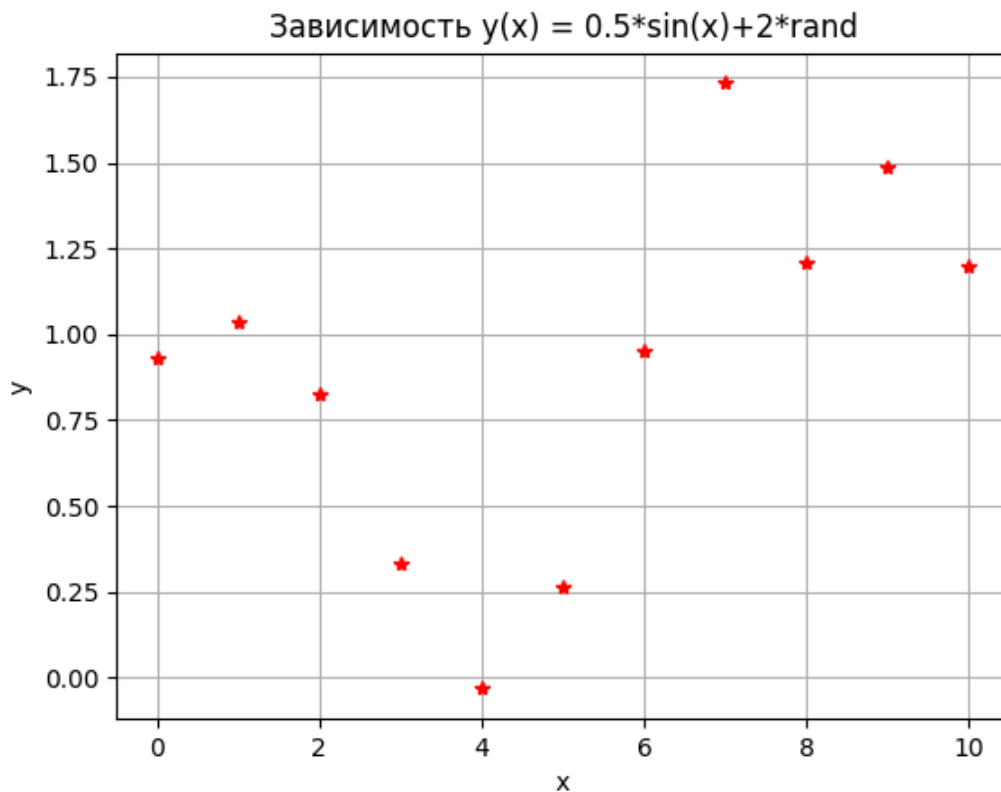
# Заполнение вектора x от 0 до 10 с шагом 1
for i in range(0, 10):
    x[i+1] = x[i] + 1

# Заполнение вектора случайных чисел и вычисление y
for i in range(0, 11):
    float_rn[i] = np.random.rand()
    y[i] = 0.5 * np.sin(x[i]) + 2 * float_rn[i]

print("Вектор значений x: \n", x)
print("Вектор сл.значений от 0 до 1: \n", float_rn)
print("Вектор значений y(x): \n", y)

# --- Часть 2: Построение графика ---

# Построение графика
plt.title("Зависимость y(x) = 0.5*sin(x)+2*rand") # заголовок
plt.xlabel("x") # ось абсцисс
plt.ylabel("y") # ось ординат
plt.grid() # включение отображение сетки
plt.plot(x, y, "r*") # построение графика (красные звездочки)
plt.show()
```



Сохранение данных в файл

```
# Сохранение исходных данных
f = open("data.txt", "w")
for i in range(0, 11):
    f.write("%f" % x[i])
    f.write(" ")
    f.write("%f \n" % y[i])

f.close()
```

Чтение данных из текстового файла

[Считываем числовые данные из файла на Python](#)

```
# Чтение исходных данных
# -----
import re
file = open("data.txt")
values = file.read().split("\n")
data = []
for key in values:
    value = re.findall(r"[-+]?[d*]\.d+|\d+", key)

    if value != []:
        data.append(value)
# -----
```

Построить интерполяционный многочлен Лагранжа

```

import matplotlib.pyplot as plt
import numpy as np

def lagrang(xf, xe, ye, ne):
    Pn = 0
    for i in range(0, ne):
        mult = 1
        for k in range(0, ne):
            if(k != i):
                mult = mult * ((xf - xe[k])/(xe[i] - xe[k]))

        Pn = Pn + ye[i] * mult

    return Pn

# Чтение исходных данных
# -----
import re
file = open("data.txt")
values = file.read().split("\n")
data = []
for key in values:
    value = re.findall(r"[-+]?[d*\.\d+|\d+", key)

    if value != []:
        data.append(value)
# -----
x = np.zeros(11)
y = np.zeros(11)
for i in range(0, 11):
    x[i] = data[i][0]
    y[i] = data[i][1]

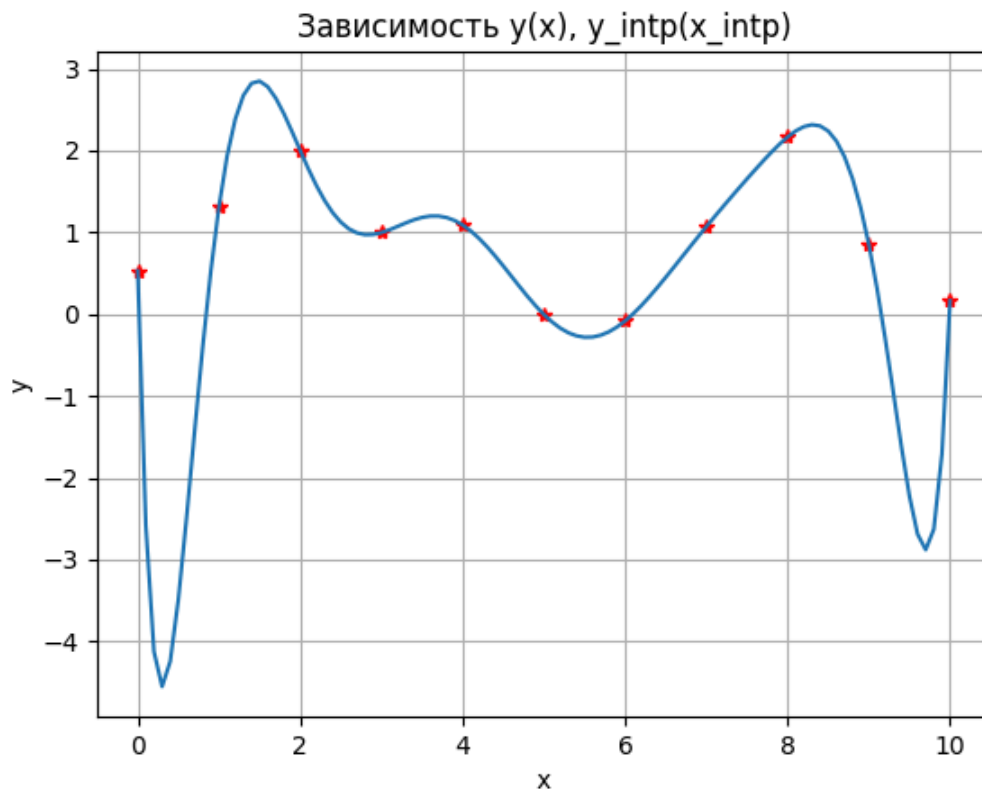
print("Считанные данные из файла: \n", data)
print("Вектор значений x: \n", x)
print("Вектор значений y(x): \n", y)

x_intp = np.zeros(101)
n_x_intp = np.size(x_intp)
for i in range(0, n_x_intp-1):
    x_intp[i+1] = x_intp[i] + 0.1

y_intp = np.zeros(n_x_intp)
n = np.size(x)
for i in range(0, n_x_intp):
    y_intp[i] = lagrang(x_intp[i], x, y, n)

# Построение графика
plt.title("Зависимость y(x), y_intp(x_intp)") # заголовок
plt.xlabel("x") # ось абсцисс
plt.ylabel("y") # ось ординат
plt.grid() # включение отображение сетки
plt.plot(x, y, "r*", x_intp, y_intp) # построение графика
plt.show()

```

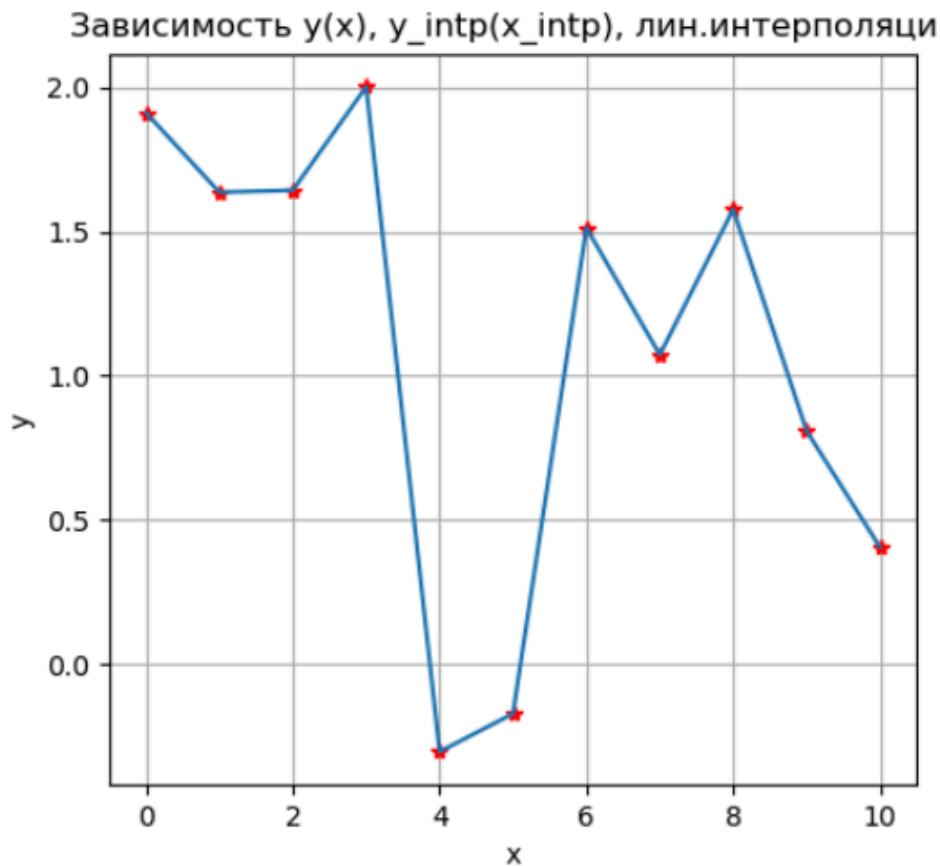


Построить линейную интерполяцию

⚠ Внимание >

К сожалению в ЛК представленный код идёт с использованием встроенных функций в библиотеки NumPy и SciPy. Тут его предоставляю пока, что в дальнейшем он будет доработан кодом собственным по реализации линейной интерполяции, кубического сплайна и МНК.

```
# Линейная интерполяция на основе библиотеки NumPy
y_intp = np.interp(x_intp, x, y)
```



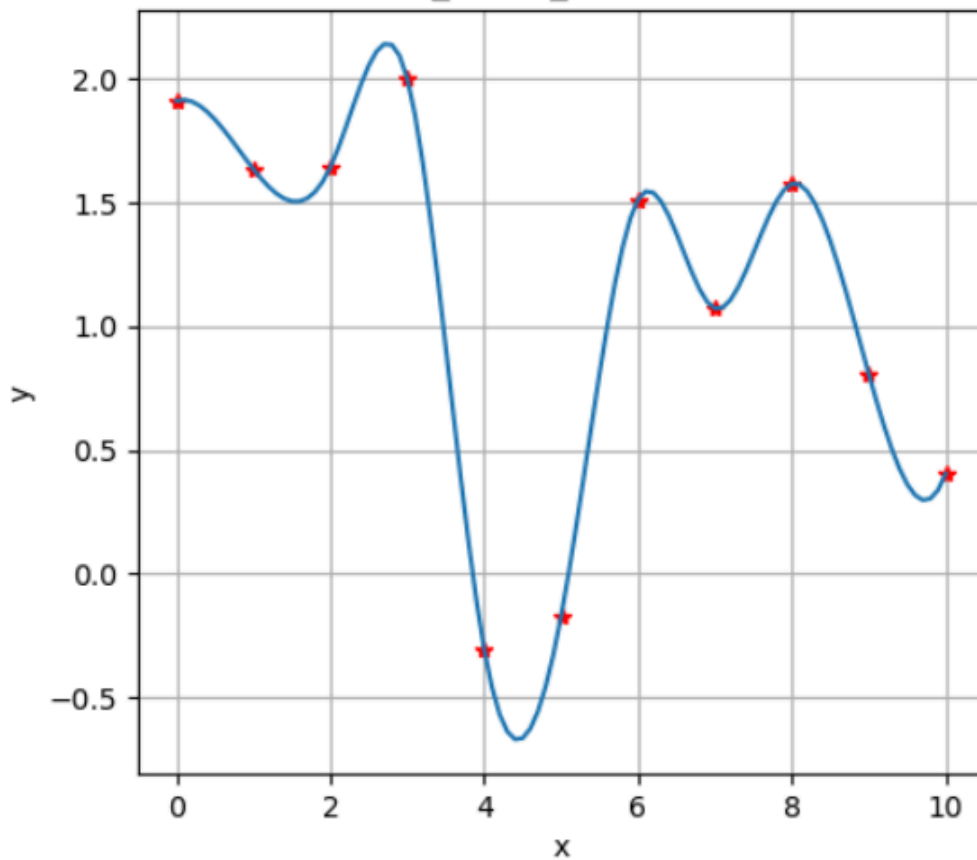
Построить кубическую сплайн-интерполяцию

⚠ Внимание >

К сожалению в ЛК представленный код идёт с использованием встроенных функций в библиотеки NumPy и SciPy. Тут его предоставляю пока, что в дальнейшем он будет доработан кодом собственным по реализации линейной интерполяции, кубического сплайна и МНК.

```
# Линейная интерполяция на основе библиотеки NumPy
y_intp = np.interp(x_intp, x, y)
```

Зависимость $y(x)$, $y_{\text{interp}}(x_{\text{interp}})$, кубическая инт.-ция

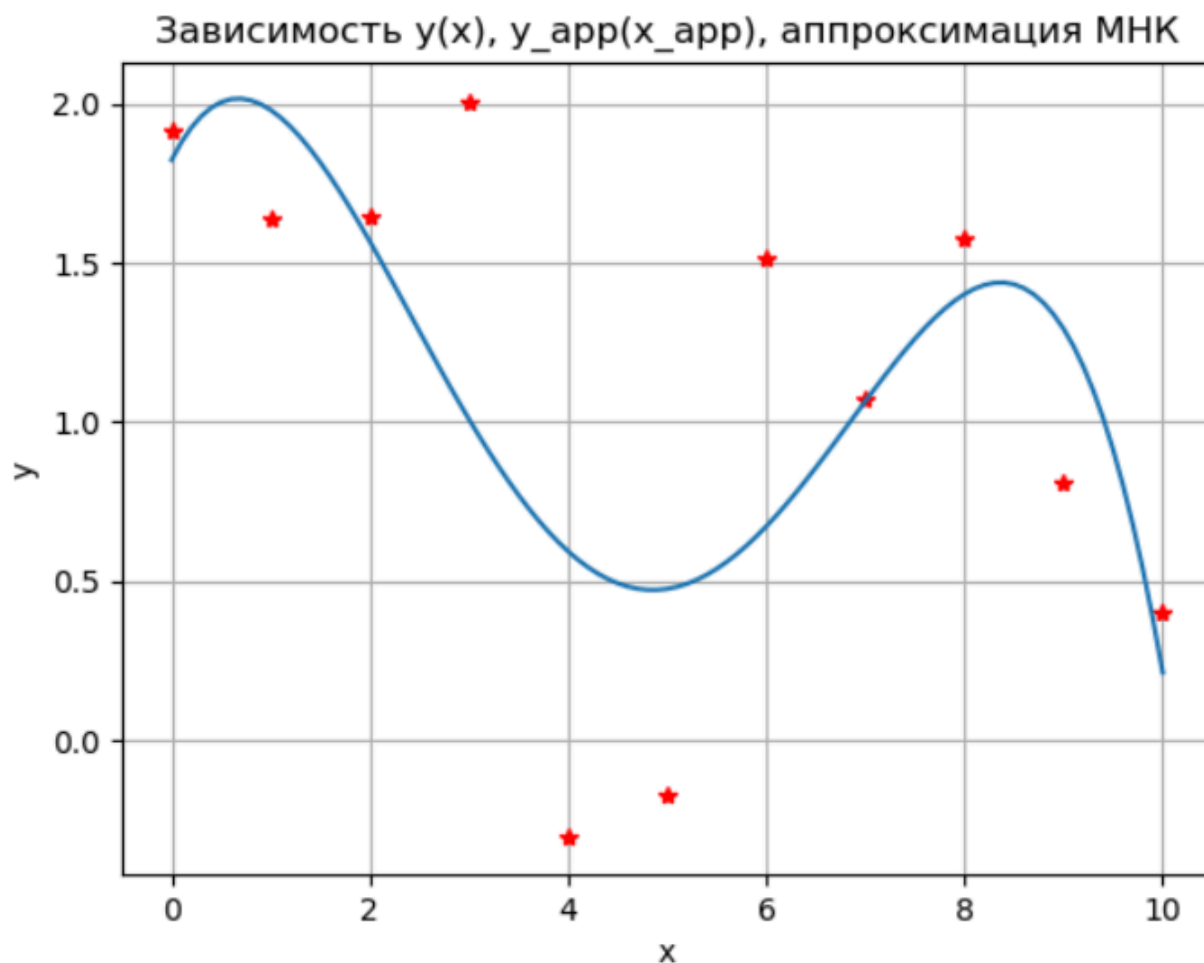


Построить аппроксимацию МНК

⚠ Внимание >

К сожалению в ЛК представленный код идёт с использованием встроенных функций в библиотеки NumPy и SciPy. Тут его предоставляю пока, что в дальнейшем он будет доработан кодом собственным по реализации линейной интерполяции, кубического сплайна и МНК.

```
# Аппроксимация данных методом наименьших квадратов
# на основе функции полифит библиотеки NumPy
coeff = np.polyfit(x, y, 4)
print("Коэффициенты полинома", coeff)
for i in range(0, n_x_app):
    y_app[i] = coeff[0] * pow(x_app[i], 4) + coeff[1] * pow(x_app[i], 3) + coeff[2] *
pow(x_app[i], 2) + coeff[3] * pow(x_app[i], 1) + coeff[4]
```



Лекция 5. Теория численного интегрирования

5.1 Задача численного интегрирования

Задача численного интегрирования заключается в вычислении определенного интеграла заданной функции $f(x)$. Характеристики численных методов:

- универсальность метода: функция $f(x)$ может задаваться в виде «черного ящика» (есть только входные и выходные значения);
- экономичность метода: количество вычислений по возможности должно быть сведено к минимуму;
- обусловленность метода: неустраняемые погрешности Δf в значениях приближенно заданной функции $f(x)$ не должны приводить к значительной итоговой ошибке ΔI .

Случаи использования:

- интегрирования функций, известных только в некоторых точках;
- интегрирования сложных выражений, не имеющих элементарных первообразных, либо имеющих слишком громоздкие выражения для них;
- построения методов численного решения уравнений в обыкновенных и частных производных (методы конечных элементов, интегро-интерполяционные методы).

5.2 Квадратурные формулы прямоугольников

Для вычисления определенного интеграла применяют приближенные равенства вида:

$$\int_a^b f(x)dx \approx \sum_{i=0}^N A_i f(\bar{x}_i)$$

где:

- $\bar{x}_i$ — узлы квадратурной формулы (точки из отрезка $[a, b]$);
- A_i — веса квадратурной формулы (числовые коэффициенты);
- $N \geq 0$ — целое число.

Сумма $\sum_{i=0}^N A_i f(\bar{x}_i)$ называется **квадратурной формулой**.

Величина

$$R = \int_a^b f(x)dx - \sum_{i=0}^N A_i f(\bar{x}_i)$$

называется **погрешностью** (или остаточным членом) квадратурной формулы.

Разбиение отрезка:

Отрезок $[a, b]$ разбивается точками $a = x_0 < x_1 < \dots < x_n = b$ на элементарные отрезки $[x_{i-1}, x_i]$.

Интеграл I разбивается на сумму элементарных интегралов:

$$I = \sum_{i=1}^n I_i$$

где $I_i = \int_{x_{i-1}}^{x_i} f(x)dx$, что соответствует разбиению исходной криволинейной трапеции на сумму площадей элементарных криволинейных трапеций.

Обозначения:

- $f_i = f(x_i)$
- $f_{i-1/2} = f(x_{i-1/2})$
- $x_{i-1/2} = (x_{i-1} + x_i)/2$ — середина элементарного отрезка
- Шаг $h = x_i - x_{i-1}$ считается постоянным.

Площадь элементарной криволинейной трапеции приближенно заменяется площадью прямоугольника с основанием $[x_{i-1}, x_i]$ и высотой $f_{i-1/2}$ (значение в средней точке).

Элементарная квадратурная формула прямоугольников:

$$I_i \approx h \cdot f_{i-1/2}$$

Составная квадратурная формула прямоугольников (центральные точки):

$$I \approx I_{\text{пп}}^h = h (f_{1/2} + f_{3/2} + \dots + f_{n-1/2}) = h \sum_{i=1}^n f_{i-1/2}$$

Выбор средней точки интервала не принципиален. Можно взять левый или правый конец интервала.

Составная квадратурная формула левых прямоугольников:

$$I \approx h \sum_{i=0}^n f_i$$

Составная квадратурная формула правых прямоугольников:

$$I \approx h \sum_{i=1}^n f_i$$

5.3 Квадратурная формула трапеций

Идея метода:

Точки $N_{i-1}(x_{i-1}, f_{i-1})$ и $N_i(x_i, f_i)$ на графике функции $y = f(x)$ соединяются отрезком. Площадь элементарной криволинейной трапеции приближенно заменяется площадью полученной трапеции.

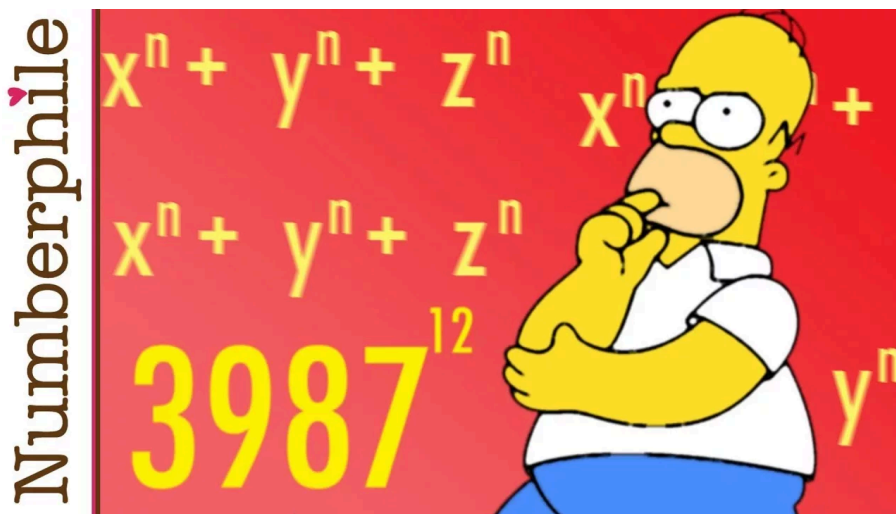
Элементарная квадратурная формула трапеций:

$$I_i \approx \frac{h}{2}(f_{i-1} + f_i)$$

Составная квадратурная формула трапеций:

$$I \approx I_{\text{тп}}^h = h \left(\frac{f_0 + f_n}{2} + \sum_{i=1}^n f_i \right)$$

5.4 Квадратурная формула Симпсона



5.5 Квадратурные формулы интерполяционного типа

Идея метода:

Подынтегральная функция f аппроксимируется на отрезке $[a, b]$ некоторой функцией g , интеграл от которой вычисляется аналитически:

$$\int_a^b f(x)dx \approx \int_a^b g(x)dx$$

Почему не глобальная аппроксимация?

Повышать точность за счет усложнения **глобальной** аппроксимации затруднительно и не всегда возможно. Поэтому используется другой подход — разбиение на элементарные отрезки.

Составная формула:

Интеграл I представляется в виде суммы интегралов по элементарным отрезкам $[x_{i-1}, x_i]$. На каждом i -м отрезке функция $f(x)$ аппроксимируется легко интегрируемой функцией $g_i(x)$:

$$\int_a^b f(x)dx \approx \sum_{i=1}^n \int_{x_{i-1}}^{x_i} g_i(x)dx$$

Вывод общей формулы:

Аппроксимируем $f(x)$ на i -м отрезке $[x_{i-1}, x_i]$ интерполяционным многочленом $P_{m,i}(x)$ с узлами:

$$z_j^{(i)} = x_{i-1/2} + t_j h_i / 2, \quad j = 0, 1, 2, \dots, m; \quad t_0, t_1, \dots, t_m \in [-1, 1]$$

Используя запись многочлена в форме Лагранжа и интегрируя $P_{m,i}$ на $[x_{i-1}, x_i]$, получаем **составную квадратурную формулу интерполяционного типа**:

$$I \approx I^h = \sum_{i=1}^n \int_{x_{i-1}}^{x_i} P_{m,i}(x) dx = \sum_{i=1}^n h_i \sum_{j=0}^m a_j f(x_{i-1/2} + t_j h_i / 2)$$

где коэффициенты a_j вычисляются как:

$$a_j = \frac{1}{2} \int_{-1}^1 \prod_{\substack{k=0 \\ k \neq j}}^m \frac{t - t_k}{t_j - t_k} dt$$

🔗 Формулы Ньютона-Котеса

Квадратурные формулы интерполяционного типа, построенные на основе **равноотстоящих** значений $t_0, t_1, \dots, t_m$, называются **формулами Ньютона-Котеса**.

Связь с ранее изученными методами:

Рассмотренные ранее простейшие квадратурные формулы являются формулами интерполяционного типа и относятся к классу формул Ньютона-Котеса:

Формула	Степень интерполяционного многочлена
Прямоугольников	нулевая
Трапеций	первая
Симпсона	вторая

5.6 Численное и символьное интегрирование в Python

Символьное интегрирование в Python

Формула: $y = 0.5 \cdot \sin(x)$, {x от 0 до 10, шаг 1}

```
import sympy as sym

x = sym.Symbol('x')
ys = 0.5*sym.sin(x)

FI = sym.integrate(ys, x)
print(FI)
Is = sym.integrate(ys, (x, 0, 10))
print(float(Is))
```

Схожим образом можно и производить символьные вычисления, код представлен на [GitHub](#).

Численное интегрирование

Полный код смотреть на [GitHub](#).

```
'''Метод прямоугольник'''
# Фрагмент
hx = 1
x = np.zeros(11)
n = np.size(x)
y = np.zeros(n)
y[0] = f(x[0])
for i in range(1,n):
    x[i] = x[i-1] + hx
    y[i] = f(x[i])
Ipr = 0
for i in range(0,n):
    Ipr = Ipr + hx * y[i]
```

```
'''Метод трапеций'''
# Фрагмент
# Схожая часть по созданию таблицы
Itr = 0
for i in range(1,n):
    Itr = Itr + (hx/2) * (y[i-1] + y[i])
```

```
'''Метод Симсона'''
# Фрагмент
# Схожая часть по созданию таблицы
St4 = 0
St2 = 0
i = 1
while(i < n):
    St4 = St4 + 4*y[i]
    i = i + 2
i = 2
while(i < n-1):
    St2 = St2 + 2*y[i]
    i = i + 2
Ism = (hx/3)*(y[0] + St4 + St2 + y[n-1])
```
